ENSC 351 Final Project Report
Autonomous UAV System

Emmett Sewell (301565227)
Devon Biernacki (301565495)
Adelina Popa (301559194)
Kian Bellinger (301554307)
Graham McRobbie (301581880)

System Overview:..... 2
Feature Table.....3
Hardware & Software/Firmware List:..... 4

System Overview:

Our project is an autonomous UAV (Unmanned Aerial Vehicle) powered by the BeagleY-AI microcontroller. Our system takes inputs from two external sensors (a LIDAR array and camera) and determines the best movement for the drone given our goal of detecting a person and maintaining a set distance from them. The system is broken down into 3 microcontrollers, the Beagle Y-AI, an STM32 F4 flight controller, and an ESP32Cam. The Beagle Y-AI is the high-level computer, running all of the detection code and handling movement commands. The flight controller handles the drone stabilization and direct movement, taking in commands from the Beagle and acting accordingly. And the ESP32 captures images from the camera and forwards them to the Beagle for human detection.

Autonomous Control: An external camera and LIDAR array provide data to control the drone. The LIDAR array is made from VL53L0X Time-of-Flight sensors, and controlled over I2C. A C program (continuousMultipleSensors.cpp) continuously polls these sensors and streams raw distance values into a Python process (imort_lidar.py). Second, the ESP32-CAM grabs JPEG frames every ~0.15 seconds and sends them to the BeagleY-AI over SPI. A python process on the BeagleY-AI runs each image through an OpenCV model (YOLOv8n) to detect humans and apply a bounding box around them (image_recog_master.py). *Image 2* in the reference images shows a sample JPEG from the OpenCV model.

Decision-making is handled by a Python script (drone_control.py) that combines data from the LIDAR array (high priority) and OpenCV (low priority). Our program monitors all LIDAR sensors, and if any return a value < 3m, a command will be sent to move the drone in the opposite direction. If all LIDAR sensors read > 3m, the camera data is given priority. Based on the size and location of the bounding box created in the last step, movement commands are sent to the flight controller to center the human within the captured frame.

Hardware Implementation: The drone was constructed using hobby-grade components. The frame was selected based on its bus-style design, giving us space to mount the components used. We decided to use 1800kv motors as this drone is not intended for high speeds. A 4S battery was used to avoid inducing higher voltage and potentially more electrical noise into the system. The flight controller and ESC chosen are a standard hobby-grade flight stack. Using these allowed us to easily mount them and connect them, without having to implement extra wires and more potential failure points. The GPS implemented is a basic GLONASS GPS, connecting to the flight controller over a UART serial port. To step down the voltage cleanly for the Beagle, a 5A 5V BEC was added, as well as various capacitors throughout the system for voltage filtering. A small voltage cell detector was also added onto the battery, giving us an alarm if the voltage dropped low. *Image 1* in the reference images displays the final drone assembly.

Extra Communication: The system uses multiple integrated communication layers. For development and monitoring, the BeagleY-AI connects to a ground-station laptop over a wireless TCP/IP link (WiFi). This lets us SSH into the Beagle from a laptop, run code, and monitor the system without tethering the drone via a physical cable. For real-time flight commands, the Beagle is hardwired to the flight controller via a UART serial connection, sending pymavlink commands to control pitch, roll, yaw, and throttle based on sensor input.

All of the source code used has been submitted along with this report, no code snippets are included here.

Image 3 in the reference images shows a complete system flowchart of all connections.

Issues and Difficulties:

Communication Issues: A major issue faced in this project was timing. The SPI communication between microcontrollers and human detection meant that there was an almost 0.5 second delay from when the frame was captured to when the inputs were sent to the drone. The solution to this would be wiring a camera directly into the BeagleY-AI, avoiding SPI communication entirely, but we were limited by available hardware.

Another major issue we faced was communication between the flight controller and the beagle over UART. Since this system was designed to be powered off a single battery, testing the connections required us to reground several components in order for the connection to be established. After testing and configuring it, the connection worked seamlessly over UART with the system powered off a single battery source.

Hardware Issues: Power distribution was also a critical issue, as we needed ample filtering throughout the system to avoid voltage spikes, potentially damaging the microcontrollers on board. The Beagle was also unable to be powered directly through its 5V pin header, therefore we had to splice apart a USB-C cable and wire it to a 5A 5V BEC, which was connected to the main battery terminals.

We also struggled with mounting all of the electronics. We were able to 3D print a case for the Beagle, however the hobby grade frame we were using was not designed to hold so many components. Mounting the ESP32 and its camera was also a challenge. However that was overcome with some ingenuity and zip ties, which actually provided a great soft mount for the camera to disrupt vibrations during flight. Our LIDAR arrays never got mounted to the drone in the end, as we did not have time to design/print custom mounts.

Software Issues: In the end, our group ran out of time to undergo actual flight testing for our drone, but all of the framework for the autonomous MavLink commands are in place. With the sensors and the human detection being the far more difficult part to implement, the drone only needs some more configuration in ArduPilot to avoid arming flags in order for it to fly. ArduPilot has a very difficult time with systems without certain sensors, for example a barometer and gps, as well as systems without a consistent RX input from a radio. These arming flags were the primary reason we were not able to get our drone in the air.

Feature Table

Description	Host/Target	Comp	Code	Authors(s)	Notes
Flight Stabilization	T	5	N/A	Emmett	Flashed F405MK2, tuned PID loops via QGC, ESC's flashed with BlHeli_S firmware and configured accordingly
Mavlink Comms	H	4	Python (PyMavlink)	Adelina, Emmett,	Heartbeat and command sending

				Kian	
LIDAR Array	H	5	Python 15% C++ 85%	Devon	Values read from lidar array over I2C (uses C++ library)
Camera Data	H & ESP32	5	Python 67% C++ 33%	Devon	ESP32 forwards JPEGS to Beagle over SPI.
Human Detection	H	5	Python (OpenCV & YOLOv8n)	Devon	Detects a human via OpenCV and creates bounding box in ~1/3s
Human Tracking & Obstacle Avoidance	H	4.5	Python	Devon	Combines Lidar & OpenCV data to control drone
Hardware mounts	Drone	2	SolidWorks	Devon	3D printed mounts for Lidar, Beagle & ESP32cam
Wireless Telemetry	H/T	5	N/A	Kian, Emmett	Wifi hotspot auto-connect. Reliable access to Beagle terminal, communication between flight controller and beagle board (UART)
ArduPilot Firmware	T	2	N/A	Kian, Emmett	Arming flags prevented proper transmission of pymavlink commands
Power Distribution, Hardware Design, Drone Construction	Hardware	5	Hardware wiring, general electronics, general mounting and construction	Emmett	Hobby grade components implemented on basis bus-style frame, high grade 5A 5V BEC for Beagle Power, mount for camera created from double sided tape and zip ties for soft mount, added capacitors for electrical noise filtering
Ground Station	N/A	5	N/A	Kian, Emmett	Set up QGroundControl on laptop for calibration, parameter setting

**For reference, the Host is the BeagleYAI, Target is the Mamba F405 flight controller.*

Hardware & Software/Firmware List:

- Flight Controller: Diatone Mamba F405 MK2 (STM32 Microcontroller) - Flashed with ArduPilot
- VL53L0X LIDAR module (Time-of-Flight)
  - Library: vl53l0x-linux (https://github.com/mjbogusz/vl53l0x-linux)
- Freenove ESP32S3-Cam board with OV2640 camera.
  - Library: OpenCV
    - Detection Model: ONNX YOLOv8n (best version of YOLO for the beagle)
- Drone frame: Martian x220
- GPS: TBS m80 GLONASS GPS
- ESC: Diatone Mamba F40 40A ESC - Flashed with BlHeli_S
- Tattu R-Line 1500mah 4s LiPo battery
- Motors: iFlight XING-E 2207 1800kv
- Accessory Components:
  - Diatone 40A Racewire
  - Generic 2-7s 5A 5V BEC
  - dLink Wifi extender (hosting private network for system)
  - LiPo cell voltage sensor

Software for Drone Control:

- QGroundControl (QGC): Essential for calibrating the accelerometer/compass and configuring SERIAL6_PROTOCOL = 2 Mavlink 2.0).
- PyMavlink: The primary Python library used to bridge the gap between our high-level logic and the drone's flight stack.
- STM32CubeProgrammer: Used to flash the initial ArduPilot bootloader onto the Mamba board.

Societal and Ethical Considerations:

The good: Creating a low-cost open-source autonomous drone means that individuals can now create solo cinematic videos at a very reasonable price. This could have major impacts on almost all major media platforms.

The bad: Creating a low-cost open-source autonomous drone with real time human detection opens up the doors for all types of surveillance/privacy violations. With a limited budget and not much time, anyone can create a drone capable of detecting and following any individual.

The ugly: This project lends itself extremely well to military applications. In recent years drones have become an integral part of warfare, and autonomous drones will become more prevalent in coming years. This project showed how easily a group could create such drone.

Reference Images:

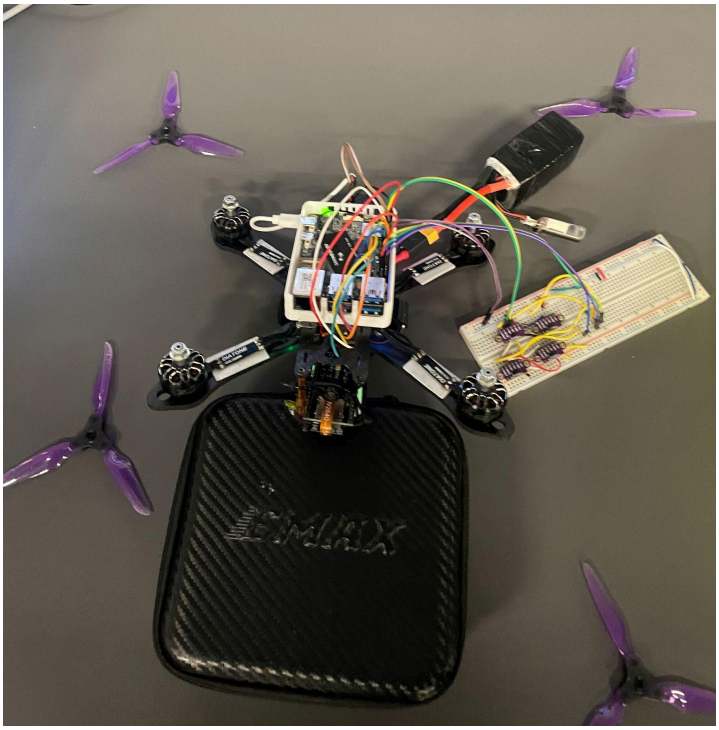

Image 1: Complete Drone Assembly

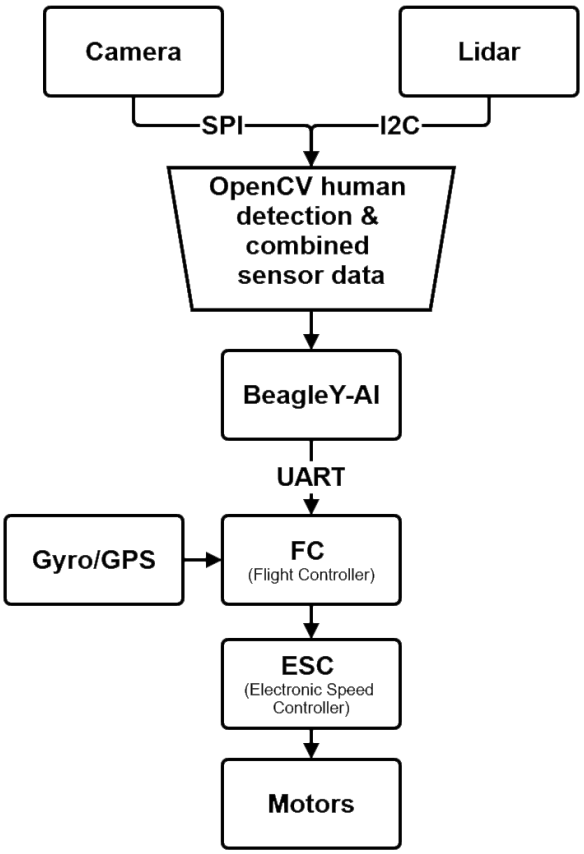

Image 2: System Flowchart

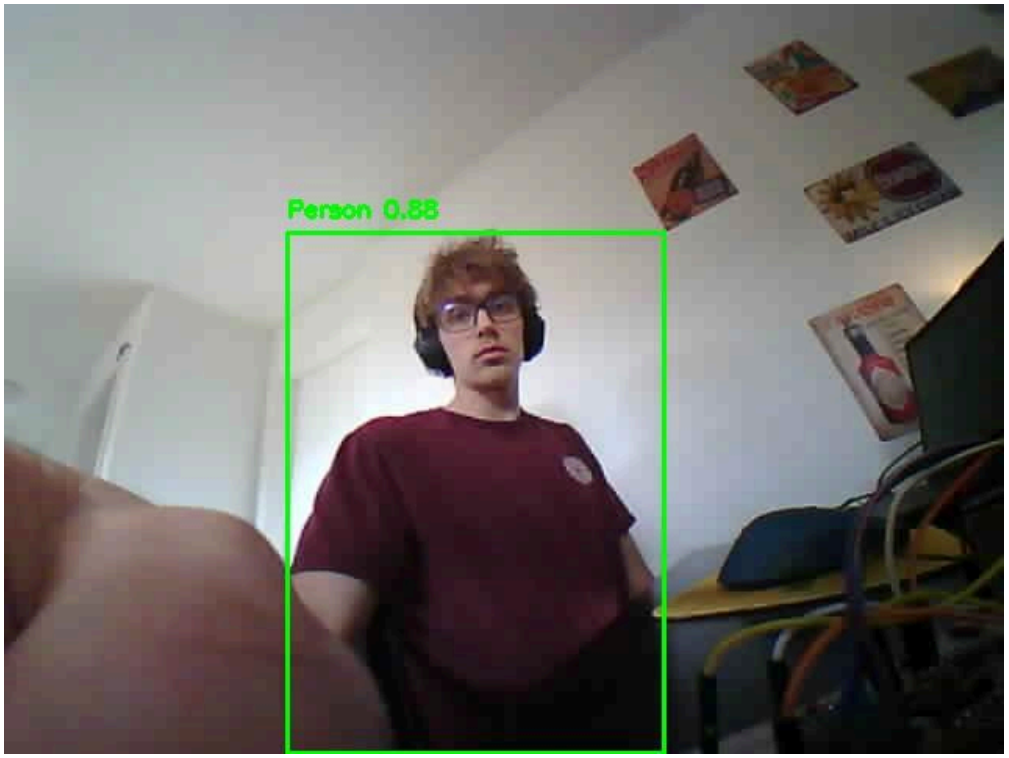

Image 3: A sample JPEG from the OpenCV Model